

# Order of Execution in SQL

Understanding the order of execution in SQL is crucial for writing efficient and correct queries. Even though you write a query in a certain order, the database engine processes it in a predefined sequence. This sequence dictates how clauses are evaluated and how the result set is progressively built.

## Standard Order of Execution

The following table outlines the standard order of execution for common SQL clauses:

| Order | Clause   | Description                                                                                                          |
|-------|----------|----------------------------------------------------------------------------------------------------------------------|
| 1     | FROM     | Identifies the table(s) or view(s) from which data will be retrieved. This is the starting point for data retrieval. |
| 2     | ON       | Specifies the join condition for tables when performing an inner join, left join, right join, or full outer join.    |
| 3     | JOIN     | Combines rows from two or more tables based on a related column between them.                                        |
| 4     | WHERE    | Filters rows based on a specified condition. Only rows that satisfy the condition are passed to the next stage.      |
| 5     | GROUP BY | Groups rows that have the same values in specified columns into summary rows. This is used for aggregate functions.  |
| 6     | HAVING   | Filters groups based on a specified condition, typically                                                             |

| Order | Clause           | Description                                                                                                         |
|-------|------------------|---------------------------------------------------------------------------------------------------------------------|
|       |                  | involving aggregate functions. It's applied after <b>GROUP BY</b> .                                                 |
| 7     | <b>SELECT</b>    | Selects the columns to be included in the final result set. Expressions and aggregate functions are evaluated here. |
| 8     | <b>DISTINCT</b>  | Removes duplicate rows from the result set. This is applied to the selected columns.                                |
| 9     | <b>ORDER BY</b>  | Sorts the result set based on one or more columns in ascending or descending order.                                 |
| 10    | <b>LIMIT/TOP</b> | Restricts the number of rows returned in the final result set.                                                      |

## Examples

Let's illustrate the order of execution with examples using a hypothetical **Orders** table and a **Customers** table.

### Example 1: Basic Query with **WHERE** and **ORDER BY**

Consider the following query: `SELECT customer_id, order_total`

`FROM Orders`

`WHERE order_date >= '2025-01-01'`

`ORDER BY order_total DESC;`

#### Execution Order:

1. **FROM Orders**: The database first identifies the **Orders** table as the source of data.

2. **WHERE order\_date >= '2025-01-01'**: Rows from the `Orders` table where the `order_date` is on or after January 1, 2025, are filtered.
3. **SELECT customer\_id, order\_total**: For the filtered rows, the `customer_id` and `order_total` columns are selected.
4. **ORDER BY order\_total DESC**: The resulting set of `customer_id` and `order_total` is then sorted in descending order based on `order_total`.

## Example 2: Query with JOIN, WHERE, GROUP BY, HAVING, and ORDER BY

Let's consider a more complex query involving multiple clauses: `SELECT c.customer_name, SUM(o.order_total) AS total_spent`

`FROM Customers c`

`JOIN Orders o ON c.customer_id = o.customer_id`

`WHERE o.order_date BETWEEN '2025-01-01' AND '2025-03-31'`

`GROUP BY c.customer_name`

`HAVING SUM(o.order_total) > 500`

`ORDER BY total_spent DESC;`

### Execution Order:

1. **FROM Customers c**: The `Customers` table is identified.
2. **JOIN Orders o ON c.customer\_id = o.customer\_id**: Rows from `Customers` and `Orders` are joined based on matching `customer_id` values.
3. **WHERE o.order\_date BETWEEN '2025-01-01' AND '2025-03-31'**: The joined rows are filtered to include only orders placed between January 1, 2025, and March 31, 2025.
4. **GROUP BY c.customer\_name**: The filtered rows are then grouped by `customer_name`. For each customer, a single group is formed.
5. **HAVING SUM(o.order\_total) > 500**: From the grouped customers, only those whose total spent (`SUM(o.order_total)`) is greater than 500 are kept.
6. **SELECT c.customer\_name, SUM(o.order\_total) AS total\_spent**: For the remaining groups, the `customer_name` and the calculated `total_spent` (sum of `order_total`) are selected.

7. **ORDER BY total\_spent DESC:** The final result set is sorted by `total_spent` in descending order.

### Example 3: Using **DISTINCT**

```
SELECT DISTINCT customer_id  
  
FROM Orders  
  
WHERE order_date >= '2025-04-01';
```

#### Execution Order:

1. **FROM Orders:** The `Orders` table is accessed.
2. **WHERE order\_date >= '2025-04-01':** Rows with an `order_date` on or after April 1, 2025, are filtered.
3. **SELECT customer\_id:** The `customer_id` column is selected from the filtered rows.
4. **DISTINCT customer\_id:** Duplicate `customer_id` values are removed, ensuring each `customer_id` appears only once in the final result.

### Example 4: Using **LIMIT** (or **TOP** for SQL Server)

```
SELECT customer_id, order_total  
  
FROM Orders  
  
ORDER BY order_total DESC  
  
LIMIT 10; -- For MySQL/PostgreSQL
```

```
-- Or TOP 10 for SQL Server: SELECT TOP 10 customer_id, order_total FROM Orders  
ORDER BY order_total DESC;
```

#### Execution Order:

1. **FROM Orders:** The `Orders` table is the source.
2. **SELECT customer\_id, order\_total:** The specified columns are selected.
3. **ORDER BY order\_total DESC:** The entire result set is sorted by `order_total` in descending order.
4. **LIMIT 10:** Only the first 10 rows from the sorted result set are returned.

Understanding this execution sequence helps in debugging queries, optimizing performance, and predicting the outcome of complex SQL statements. It ensures you structure your queries logically to achieve the desired results.